

Un viaje breve por los tipos de algoritmos

Bryan Ezequiel Violante Arriaga

Prope. Programación y Estructuras de Datos

17 de mayo de 2026

Resumen

El presente trabajo (correspondiente a la actividad 3 del curso propedéutico de Programación y Estructuras de Datos para la MC. en Ciencias de la Computación) recorre los distintos tipos de algoritmos que suelen estudiarse en un curso de análisis y diseño. Tras unas notas sobre la noción de algoritmo y la medición de su eficiencia mediante notación asintótica, se abordan las familias clásicas: determinísticos y no determinísticos, divide y vencerás, voraces, programación dinámica, vuelta atrás (backtracking), ramificación y acotación, probabilísticos, paralelos y metaheurísticas. Para cada una se comenta su idea principal, los problemas en los que conviene aplicarla y, cuando resulta útil, un pequeño ejemplo en pseudocódigo. Al cierre se incluye una tabla comparativa y algunas reflexiones sobre cuándo elegir una técnica u otra.

1. Introducción

Resolver problemas con una computadora implica, básicamente, diseñar un algoritmo, que en palabras simples es una receta paso a paso que recibe una entrada y produce una salida (similar a una receta de cocina, por ejemplo). Tareas como ordenar una lista, calcular la ruta más corta entre dos ciudades, decidir si un número es primo o planificar tareas en una fábrica son ejemplos cotidianos para un científico de la computación. La buena noticia es que, con el paso del tiempo, las personas dedicadas a esto han identificado varios *patrones* de diseño que resuelven familias enteras de problemas. La mala noticia es que no hay un único patrón que sirva para todo.

En este trabajo se recorren los patrones más conocidos y trata de responder, de manera práctica y concisa, tres preguntas para cada uno: ¿en qué consiste?, ¿qué tipo de problemas resuelve bien? y ¿cuánto cuesta usarlo?

2. Marco teórico

2.1. ¿Qué es un algoritmo?

Como se mencionó, se puede pensar en un algoritmo como una receta de cocina: más formalmente, un algoritmo es una secuencia finita de instrucciones bien definidas que transforma una entrada en una salida. Las propiedades clave que suele exigirse son cinco:

- **Finitud:** termina en un número finito de pasos.
- **Definitud:** cada paso es claro y sin ambigüedad.
- **Entrada:** recibe cero o más datos.
- **Salida:** produce uno o más resultados.
- **Efectividad:** cada operación es realizable.

2.2. Complejidad

La eficiencia de un algoritmo se mide observando cuántas operaciones realiza en función del tamaño n de la entrada. A esta cuenta se le llama *complejidad temporal*, $T(n)$. De manera similar, la *complejidad espacial* mide la memoria que necesita.

2.3. Notación asintótica

Para describir el crecimiento de $T(n)$ se usan tres notaciones:

- **Cota superior:** $f(n) = \mathcal{O}(g(n))$ si existen constantes $c > 0$ y n_0 tales que

$$0 \leq f(n) \leq c g(n), \quad \forall n \geq n_0.$$

- **Cota inferior:** $f(n) = \Omega(g(n))$ si existen $c > 0$ y n_0 tales que

$$0 \leq c g(n) \leq f(n), \quad \forall n \geq n_0.$$

- **Cota ajustada:** $f(n) = \Theta(g(n))$ si se cumplen ambas a la vez.

Un algoritmo es *polinómico* si $T(n) = \mathcal{O}(n^k)$ para alguna constante k , y *exponencial* si $T(n) = \mathcal{O}(c^n)$ con $c > 1$. A simple vista (al menos para el autor) resultaría que no hay mucha diferencia, pero esto no es así, ya que la diferencia es enorme en la práctica.

A continuación se muestra una tabla comparativa, en ella se destaca el tiempo de ejecución para diferentes tamaños de entrada y diferentes tiempos de ejecución, suponiendo que el procesador puede realizar 10^9 operaciones por segundo (siendo modestos pues es común que los equipos de hoy en día superen esa cifra).

Tabla 1: Tiempo de ejecución (orden de magnitud) suponiendo 10^9 operaciones por segundo.

$T(n)$	$n = 10$	$n = 30$	$n = 50$
n	~ 10 ns	~ 30 ns	~ 50 ns
n^2	~ 0.1 μ s	~ 1 μ s	~ 2.5 μ s
n^5	0.1 ms	~ 24.3 ms	~ 0.3 s
2^n	1 μ s	~ 1 s	≈ 13 días
3^n	59 μ s	≈ 57 hrs	≈ 22 millones de años

2.4. Clases de problemas

De manera informal, un problema está en la clase **P** si existe un algoritmo polinómico que lo resuelve, y está en **NP** si una solución propuesta puede verificarse en tiempo polinómico. Los problemas **NP**-duros son los más complejos dentro de **NP**. Si estos problemas tienen o no algoritmos polinómicos es la famosa pregunta abierta $P \stackrel{?}{=} NP$.

3. Algoritmos determinísticos

Un algoritmo es *determinístico* cuando, para una misma entrada, su comportamiento es siempre el mismo: cada paso lleva a un único siguiente paso y la salida está completamente determinada por la entrada. Es el tipo de algoritmo más común y, en cierto sentido, el “predeterminado” que pensamos al momento de pensar en un algoritmo para resolver un problema específico; hacer A, luego B, y así sucesivamente.

Este tipo de algoritmo puede servir como solución o enfoque correcto para muchos problemas, prácticamente todos los algoritmos clásicos que se estudian al inicio entran aquí (búsqueda secuencial, ordenamiento por inserción, búsqueda en profundidad, etc.). Aparece sobre todo por contraste con las otras categorías ya que en la mayoría de los casos, la solución otorgada por este tipo de algoritmos no es la mejor posible.

La ejecución de este tipo de algoritmos corresponde a una función $f : \mathcal{I} \rightarrow \mathcal{O}$ totalmente definida. En palabras simples, si se ejecuta dos veces con la misma entrada, se obtiene exactamente la misma salida y se pasa por los mismos pasos (de manera similar a un AFD).

Un ejemplo clásico de algoritmo determinístico es la búsqueda secuencial, que recorre un

arreglo de izquierda a derecha hasta encontrar el valor buscado. En el peor caso, recorre todo el arreglo, lo que da una complejidad de $\Theta(n)$. El pseudocódigo es el siguiente:

```
busqueda_secuencial(A, x):  
    # A es un arreglo de tamaño n, x el valor buscado  
    para i = 1, 2, ..., n:  
        si A[i] == x:  
            devolver i  
    devolver -1    # no se encontro
```

Puede observarse que el algoritmo busca de manera ingenua, sin aprovechar ninguna propiedad de la entrada, esto lo hace poco eficiente para arreglos grandes, y por supuesto existen mejores algoritmos para este tipo de problemas, los cuales se estudiarán más adelante.

4. Algoritmos no determinísticos

Un algoritmo *no determinístico* se ejecuta de manera similar a un árbol: en cada paso el cómputo puede bifurcarse en varios sucesores y todas las ramas se ejecutan, en teoría, al mismo tiempo. Se considera que el algoritmo acepta si *alguna* de las ramas llega a un estado de aceptación (resulta una analogía muy interesante con los AFN y la conversión a AFD pues en este proceso se exploran todas las posibles transiciones, formando un árbol).

Este tipo de algoritmos permiten definir formalmente la clase **NP**: un problema está en **NP** si existe una máquina no determinística que lo resuelve en tiempo polinómico. Cuando hay que “simular” este comportamiento en una computadora normal, lo que se hace es recorrer ese árbol de posibilidades con técnicas como *backtracking* o ramificación y acotación, las cuales se estudiarán en las secciones posteriores.

El principio de este algoritmo parecería simple, pero es muy poderoso, en cada paso, en lugar de elegir una transición, el algoritmo “adivina” la correcta. La pregunta abierta más importante de la teoría de la computación, $P = NP$, pregunta si toda esa magia adivinatoria puede sustituirse por un algoritmo determinístico polinómico.

5. Divide y vencerás

En los anales de la historia, se le atribuye al general romano Julio César la frase “divide y vencerás”, la cual consistía en dividir las fuerzas enemigas en concentraciones más pequeñas para derrotarlas de manera más fácil.

Esta técnica se basa en ese concepto antiquísimo y consiste en, dado un problema, partirlo en pedazos más pequeños del mismo tipo, resolver cada uno por separado (normalmente con la misma técnica, de forma recursiva) y combinar los resultados. Los tres pasos clásicos son: **dividir**, **vencer** (resolver los subproblemas) y **combinar**.

Funciona muy bien cuando el problema admite una partición natural en partes independientes y de tamaño parecido. Los ejemplos clásicos incluyen merge sort (*merge sort*), búsqueda binaria, multiplicación rápida de enteros grandes y el algoritmo de Strassen para multiplicar matrices.

Si un algoritmo divide y vencerás genera a subproblemas de tamaño n/b y emplea $f(n)$ tiempo en dividir y combinar, su tiempo de ejecución satisface

$$T(n) = aT\left(\frac{n}{b}\right) + f(n).$$

El *teorema maestro* permite resolver estas recurrencias de manera sistemática (de manera muy simplificada, este teorema dice que basta con comparar $f(n)$ con $n^{\log_b a}$, el costo acumulado de los subproblemas y, según cuál de los dos domine, entrega directamente la forma cerrada de $T(n)$ sin necesidad de desarrollar el árbol de recursión a mano).

A continuación se muestra el pseudocódigo del algoritmo de merge sort, que divide el arreglo a ordenar en dos mitades, las ordena recursivamente y luego las fusiona en un arreglo ordenado. La función MERGE se encarga de combinar las dos mitades ya ordenadas, y su complejidad es $\Theta(n)$.

```
merge_sort(A, p, r):  
    # A es el arreglo, p y r los índices del subarreglo a ordenar  
    si p < r:  
        q = (p + r) // 2  
        merge_sort(A, p, q)  
        merge_sort(A, q+1, r)  
        merge(A, p, q, r)    # fusiona las dos mitades ya ordenadas
```

Se puede ver que la recurrencia $T(n) = 2T(n/2) + \Theta(n)$ da $T(n) = \Theta(n \log n)$, que es lo mejor que puede hacerse para ordenamiento basado en comparaciones.

6. Algoritmos voraces (*greedy*)

En la vida diaria, al estar hambriento y, presentarse diversos platillos para comer, se tiende a elegir lo más apetecible, al terminar, se elige el siguiente platillo más apetecible de entre las opciones restantes y así sucesivamente. De manera similar, un algoritmo voraz construye la solución paso a paso: en cada momento elige el candidato que se ve más prometedor según un criterio local, y no se vuelve atrás. Es la estrategia del que va decidiendo “lo mejor que se ve ahora”, con la esperanza de llegar a un buen resultado global.

Este enfoque se aplica a problemas de optimización. Funciona perfectamente cuando el problema cumple dos propiedades: **subestructura óptima** (la solución óptima del problema contiene soluciones óptimas de sus subproblemas) y **elección voraz** (existe una decisión local que pertenece a alguna solución óptima).

Ejemplos clásicos donde sí funciona: árbol de recubrimiento mínimo (Kruskal, Prim), códigos de Huffman, camino más corto desde un origen (Dijkstra), selección de actividades.

Es importante aclarar que en muchos problemas la estrategia voraz NO da el resultado óptimo. En el problema del agente viajero, por ejemplo, ir siempre al vecino más cercano da rutas que pueden ser mucho peores que la mejor posible.

Un algoritmo voraz suele tener cinco partes:

- El conjunto de candidatos.
- Los conjuntos de seleccionados y rechazados.
- Una función de factibilidad (¿se puede seguir armando una solución?).
- Una función de selección (¿cuál es el mejor candidato disponible?).
- Una función objetivo (¿cuándo se ha alcanzado una solución completa?).

A continuación se muestra un ejemplo de algoritmo voraz: el algoritmo de Kruskal, que construye un árbol de recubrimiento mínimo (MST) eligiendo en cada paso la arista de menor peso que no forme ciclo.

```
kruskal(G, w):  
    # G = (V, E) grafo conexo, w: E -> R+ peso de cada arista  
    T = {}                                # conjunto de aristas del MST  
    para cada v en V:  
        make_set(v)                       # cada vertice en su propio componente
```

```

ordenar E por peso w ascendente
para cada arista (u, v) en E (en orden):
    si find_set(u) != find_set(v):
        T = T union {(u, v)} # no forma ciclo, se acepta
        union(u, v)
devolver T

```

7. Programación dinámica

Al igual que divide y vencerás, la programación dinámica también descompone el problema en subproblemas, pero a diferencia de este, los subproblemas *se repiten*. La idea entonces es resolver cada subproblema una sola vez y guardar el resultado en una tabla para reutilizarlo. Se cambia memoria por tiempo, y a veces el ahorro es enorme: un algoritmo recursivo ingenuo exponencial puede volverse polinómico con esta técnica.

Este tipo de algoritmos funciona muy bien cuando el problema cumple dos características muy específicas:

1. **Subestructura óptima:** la solución óptima del problema se construye con soluciones óptimas de subproblemas.
2. **Subproblemas solapados:** los mismos subproblemas aparecen una y otra vez.

Ejemplos típicos: mochila 0-1, distancia de edición, parentización óptima de productos de matrices, problema del cambio de moneda, alineamiento de secuencias en bioinformática.

Los algoritmos basados en programación dinámica suelen seguir este patrón/esquema general:

1. Se caracteriza cómo se ve una solución óptima.
2. Se define recursivamente el valor de una solución óptima.
3. Se calcula ese valor, normalmente *de abajo hacia arriba* (*bottom-up*) aunque también se puede hacer *de arriba hacia abajo* (*top-down*) con memoización.
4. Se reconstruye la solución óptima a partir de la información guardada.

Y hay dos formas equivalentes de implementarlo: la *memoización* (recursión que guarda resultados) y la *tabulación* (iterativo, llenando la tabla).

En la experiencia del autor, la programación dinámica es de las técnicas más difíciles de

entender al inicio y de dominar, sobre todo por la necesidad de identificar la estructura de los subproblemas y la relación entre ellos. Sin embargo, una vez que se entiende, es una herramienta extremadamente poderosa.

A continuación se muestra un ejemplo de algoritmo de programación dinámica (meramente ilustrativo); el problema del cambio de moneda, el cual consiste en, dada una cantidad N y un conjunto de denominaciones $\{d_1, \dots, d_n\}$, encontrar el número mínimo de monedas para pagar N . Sea $c[i, j]$ el número mínimo de monedas para pagar j usando denominaciones $\{d_1, \dots, d_i\}$. La recurrencia es:

$$c[i, j] = \min(c[i-1, j], 1 + c[i, j - d_i]).$$

Y el pseudocódigo es el siguiente:

```

cambio_monedas(d, N):
    # d = denominaciones [d_1, ..., d_n], N = cantidad objetivo
    # c[i][j] = monedas minimas para formar j usando solo d[1..i]
    para i = 1, ..., n:
        c[i][0] = 0                # 0 monedas para cantidad 0
    para i = 1, ..., n:
        para j = 1, ..., N:
            si i == 1 y j < d[i]:
                c[i][j] = +infinito    # no alcanzable
            si no, si i == 1:
                c[i][j] = 1 + c[i][j - d[i]]
            si no, si j < d[i]:
                c[i][j] = c[i-1][j]    # no cabe la moneda d[i]
            si no:
                c[i][j] = min( c[i-1][j],          # no uso d[i]
                               1 + c[i][j - d[i]] ) # uso d[i]
    devolver c[n][N]

```

La complejidad de este algoritmo es $\Theta(nN)$ (pseudopolinómica).

8. Vuelta atrás (*backtracking*)

El *backtracking* explora el espacio de soluciones como si fuera un árbol: va construyendo una solución parcial paso a paso, y cuando descubre que esa rama no puede llevar a una solución válida, retrocede y prueba otra. Es básicamente una búsqueda en profundidad con poda por

factibilidad.

Cualquier problema donde la solución se pueda expresar como una secuencia de decisiones $(x_1, x_2, \dots, x_k)$ con restricciones que se verifican sobre la marcha. Algunos ejemplos clásicos son: las n -reinas, suma de subconjuntos, coloreo de grafos, generación de permutaciones, sudoku.

A continuación se muestra el esquema general de un algoritmo de backtracking, que recibe una solución parcial $\mathbf{x} = (x_1, \dots, x_k)$ y el nivel k del árbol. Si $\mathbf{x}$ es una solución completa, se procesa (por ejemplo, se imprime). Si no, se generan los candidatos para x_{k+1} y se verifica su factibilidad.

```
backtrack(x, k):
    # x = (x_1, ..., x_k) es la solución parcial construida hasta ahora
    si x es una solución completa:
        procesar(x)
    si no:
        para cada candidato v para x_{k+1}:
            si (x, v) es factible:
                x_{k+1} = v
                backtrack(x, k+1)
            # al volver, se descarta v y se prueba otro
```

En el peor caso es exponencial, pero la poda por factibilidad suele recortar de manera significativa el árbol. Para las 8-reinas, por ejemplo, se examinan cerca de 1.3 millones de configuraciones en lugar de los más de 16 millones del enfoque más obvio.

9. Ramificación y acotación (*branch and bound*)

La ramificación y acotación es como un *backtracking* con cerebro: además de descartar ramas por factibilidad, calcula una *cota* (superior o inferior, según se busque máximo o mínimo) del mejor valor posible alcanzable desde cada nodo. Si esa cota ya es peor que la mejor solución conocida, no tiene sentido seguir explorando esa rama y se poda.

Este tipo de algoritmos son muy buenos para resolver problemas de optimización combinatoria NP-duros para los que aún así se quiere la solución óptima exacta:

- **Mochila 0-1:** dado un conjunto de objetos con peso y valor, y una capacidad máxima, se busca el subconjunto de objetos que maximice el valor sin exceder la capacidad.

- **Programación entera:** optimización lineal con la restricción de que algunas o todas las variables deben ser enteras.
- **Asignación cuadrática:** asignar n tareas a n agentes minimizando un costo que depende de las asignaciones.
- **Agente viajero:** encontrar la ruta más corta que visite un conjunto de ciudades exactamente una vez y regrese al punto de partida.

Suele usarse en instancias medianas que el *backtracking* puro no podría manejar.

En general el peor caso sigue siendo exponencial, pero en la práctica las podas hacen que instancias razonables se resuelvan en tiempos manejables.

10. Algoritmos probabilísticos

Un algoritmo *probabilístico* (también llamado aleatorizado) toma al menos una decisión en función de números pseudoaleatorios. Para una misma entrada, dos ejecuciones pueden seguir caminos distintos e incluso, en algunos casos, dar respuestas distintas. Suena raro al inicio, pero introducir aleatoriedad puede ser muy útil: a veces simplifica el algoritmo, otras lo hace asintóticamente más rápido y otras evita los casos peores que un algoritmo determinístico tendría. Es como si se introdujera entropía para escapar de la rigidez de los algoritmos determinísticos.

Se suelen distinguir tres familias:

- **Monte Carlo.** Siempre terminan en un tiempo acotado, pero pueden dar una respuesta incorrecta con cierta probabilidad. Esta probabilidad se puede reducir tanto como se quiera repitiendo el algoritmo. Ejemplo clásico: el test de primalidad de Miller–Rabin.
- **Las Vegas.** Nunca dan una respuesta incorrecta, pero su tiempo de ejecución es aleatorio. Ejemplo: *Quicksort* con pivote aleatorio, cuyo tiempo esperado es $\Theta(n \log n)$.
- **Numéricos.** Producen una aproximación cuya precisión mejora con el tiempo de cómputo. Ejemplo típico: la integración por Monte Carlo, útil sobre todo para integrales en muchas dimensiones.

A continuación, se muestra un ejemplo de cómo se puede mejorar la probabilidad de acierto de un algoritmo de Monte Carlo mediante repeticiones (el cual pretende ser meramente ilustrativo, no se pretende el análisis riguroso):

Supóngase que se tiene un algoritmo de Monte Carlo que acierta con probabilidad $\geq 1/2 + \varepsilon$. Si se ejecuta k veces (con k impar) y se devuelve la respuesta más frecuente, la probabilidad de equivocarse cae exponencialmente con k .

```

repetir_MC(x, k):
    # x = instancia, k = numero impar de repeticiones
    # MC(x) es el algoritmo Monte Carlo base (puede equivocarse)
    V = 0    # votos a "verdadero"
    F = 0    # votos a "falso"
    para i = 1, ..., k:
        si MC(x) == verdadero:
            V = V + 1
        si no:
            F = F + 1
    si V > F:
        devolver verdadero
    si no:
        devolver falso

```

11. Algoritmos paralelos

Suponga que se tiene una serie de tareas las cuales se pueden ejecutar al mismo tiempo, ¿convendría hacerlas secuencialmente o en paralelo?. Un algoritmo *paralelo* toma este principio y divide el trabajo entre varios procesadores que actúan al mismo tiempo. Los objetivos son los de siempre que se piensa en paralelo: terminar antes, atacar problemas más grandes y, en algunos casos, ser más robustos ante fallas.

La clasificación clásica para este tipo de algoritmos (Flynn) distingue:

- **SISD** (*Single Instruction, Single Data*): el cómputo secuencial de toda la vida.
- **SIMD** (*Single Instruction, Multiple Data*): la misma instrucción se aplica sobre datos distintos. Es lo que hacen las GPU.
- **MIMD** (*Multiple Instruction, Multiple Data*): cada procesador puede hacer cosas distintas con datos distintos. Es lo común en clústeres y procesadores multinúcleo.

Dos dominan la práctica:

- **Memoria compartida:** todos los procesos ven la misma memoria. Se programa con hilos (Pthreads, OpenMP, hilos de Java).
- **Paso de mensajes:** los procesos no comparten memoria y se comunican mandándose mensajes. Lo típico en clústeres y *grids*; el estándar es MPI.

Para medir el rendimiento de un algoritmo paralelo, se utilizan dos métricas principales. Si T_1 es el tiempo del algoritmo secuencial y T_p el tiempo con p procesadores, se definen:

$$\text{Speedup: } S_p = \frac{T_1}{T_p}, \quad \text{Eficiencia: } E_p = \frac{S_p}{p}.$$

La famosa *ley de Amdahl* pone un techo a este *speedup* en función de la fracción del algoritmo que no se puede paralelizar.

Por ejemplo, para sumar n enteros (con n potencia de 2) usando muchos procesadores, se organizan los datos como hojas de un árbol binario completo. En cada nivel los procesadores suman pares en paralelo. El resultado se obtiene en $\Theta(\log n)$ pasos en lugar de los $\Theta(n)$ que toma el algoritmo secuencial.

```
suma_paralela(A):
    # A es un arreglo de tamaño n = 2^k
    # en cada nivel se suman parejas vecinas en paralelo
    para l = 1, ..., log2(n):
        para i = 1, ..., n / 2^l    EN PARALELO:
            A[i] = A[2i - 1] + A[2i]
    devolver A[1]
```

12. Metaheurísticas

La etimología de la palabra *metaheurística* proviene de las raíces griegas *meta* (“más allá”) y *heuriskein* (“descubrir”): es decir, una técnica que va más allá de las heurísticas tradicionales, actuando como una estrategia general que guía o modifica otras heurísticas para explorar el espacio de soluciones de manera más efectiva.

A diferencia de los algoritmos exactos, las metaheurísticas no garantizan que la solución encontrada sea óptima. Pero en muchos problemas reales esto no es un problema como tal, a cambio se consiguen muy buenas soluciones en tiempos razonables, que es justo lo que se necesita en la práctica.

Es especialmente útil en problemas de optimización combinatoria y continua de gran escala:

- **Combinatoria:** TSP, ruteo de vehículos, asignación de tareas, programación entera, etc.
- **Continua:** optimización de funciones no convexas, ajuste de parámetros en modelos complejos, etc.
- **Híbridos:** problemas que combinan elementos discretos y continuos, como el diseño de redes o la planificación de la producción.

entre muchos otros. En general, problemas NP-duros donde se necesita una buena respuesta pronto.

Toda metaheurística tiene que balancear dos cosas:

- **Exploración (diversificación):** cubrir muchas regiones distintas del espacio de búsqueda para no quedarse atrapada en un óptimo local.
- **Explotación (intensificación):** refinar la búsqueda en las regiones más prometedoras para acercarse al mejor valor posible.

Lograr el balance adecuado es, en gran medida, el arte de diseñar una buena metaheurística.

Y se suelen dividir en dos grandes familias:

1. **Basadas en una solución** (orientadas a la explotación): *hill climbing*, *simulated annealing*, búsqueda tabú, búsqueda local iterada (ILS), GRASP, búsqueda en vecindad variable (VNS).
2. **Basadas en población** (orientadas a la exploración): algoritmos genéticos y, más en general, algoritmos evolutivos, estrategias de evolución, optimización por enjambre de partículas (PSO), colonias de hormigas (ACO), búsqueda dispersa, sistemas inmunes artificiales.

Algo interesante es que el análisis asintótico clásico no dice mucho de las metaheurísticas, y esto es esencial en la práctica porque obtienen una solución muy buena en tiempos razonables. Por eso suelen evaluarse experimentalmente, sobre instancias de prueba, comparando estadísticamente con otros métodos.

13. Análisis comparativo

A continuación se muestra la tabla 2 que resume las propiedades principales de cada técnica. Se contrastan complejidad temporal típica, uso de memoria, si garantizan o no la solución óptima y dónde suelen usarse.

Tabla 2: Comparativa de las principales técnicas de diseño algorítmico.

Técnica	Complejidad	Memoria	¿Óptimo?	Aplicaciones
Divide y vencerás	$\mathcal{O}(n \log n)$ a $\mathcal{O}(n^c)$	$\mathcal{O}(\log n)$ (pila)	Sí	Mergesort, búsqueda binaria, Strassen
Voraces	$\mathcal{O}(n \log n)$ típico	Baja	Solo con subestr. óptima	MST, Huffman, planificación
Programación dinámica	Polinómica o pseudopolinómica	$\Theta(\text{tabla})$, alto	Sí	Mochila 0-1, distancia de edición
Backtracking	Exponencial (peor caso)	$\mathcal{O}(n)$ (pila)	Sí (exhaustivo)	n -reinas, coloración
Ramif. y poda	Exponencial; mejor con poda	Variable	Sí	Mochila 0-1, TSP
Probabilísticos	Las Vegas: variable; Monte Carlo: acotada	Baja	Probabilística	Miller–Rabin, Quicksort aleatorio
Paralelos	Reducción por factor $\leq p$ (Amdahl)	Depende del modelo	Igual al base	Cómputo científico, IA
Metaheurísticas	Según presupuesto	Variable	No	Optimización NP-difícil

13.1. ¿Cómo elegir?

Después de haber visto todos los algoritmos y una breve comparación sobre ellos, surge la pregunta natural, ¿cómo elegir la técnica adecuada? Bueno, la elección de la técnica depende, en la práctica, de cuatro cosas:

1. **Naturaleza del problema.** Si admite subestructura óptima, voraces o programación dinámica son buenos candidatos. Si requiere explorar combinaciones, *backtracking* o ramificación y acotación.
2. **Tamaño de la entrada.** Para entradas pequeñas, los algoritmos exactos exponenciales son perfectamente usables. Para entradas grandes en problemas difíciles, prácticamente no queda más opción que las metaheurísticas.
3. **Recursos disponibles.** La programación dinámica gasta mucha memoria, el paralelismo requiere *hardware* adecuado, las metaheurísticas requieren tiempo de ajuste de parámetros.
4. **Garantías requeridas.** Si se necesita el óptimo exacto, los métodos aproximados quedan descartados. Si basta con “muy bueno”, se abre todo el abanico.

Aunque es solo teoría, en la práctica casi nunca se usa una técnica pura. Lo común es combinar:

por ejemplo, una metaheurística que arranca con una heurística voraz para tener una buena solución inicial, usa programación dinámica para resolver exactamente algunos subproblemas y aprovecha varios núcleos en paralelo. La hibridación es más la regla que la excepción.

14. Conclusiones

A lo largo del trabajo se revisaron las principales familias de algoritmos. Las ideas más importantes que conviene llevarse son:

Primero, los algoritmos se pueden clasificar por más de un criterio: por su comportamiento (determinístico, no determinístico, probabilístico), por la estructura del cómputo (secuencial o paralelo) o por la técnica de diseño (divide y vencerás, voraz, programación dinámica, vuelta atrás, ramificación y acotación). Estas dimensiones no se excluyen: un mismo algoritmo puede pertenecer a varias categorías a la vez.

Y segundo, no hay una técnica universalmente mejor. La elección depende del problema concreto, del tamaño de las entradas, de los recursos disponibles y de qué garantías se necesitan sobre la solución.

15. Bibliografía

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. *Introduction to Algorithms*. 3.^a ed. The MIT Press, Cambridge, Massachusetts, 2009.
2. Brassard, G. y Bratley, P. *Fundamentos de Algoritmia*. Prentice Hall, Madrid, 1997. Traducción del original publicado por el Département d'informatique et de recherche opérationnelle, Université de Montréal.
3. Sipser, M. *Introduction to the Theory of Computation*. 3.rd ed. Cengage Learning, Boston, Massachusetts, 2012.
4. Talbi, E.-G. *Metaheuristics: From Design to Implementation*. John Wiley & Sons, Hoboken, Nueva Jersey, 2009. University of Lille - CNRS - INRIA.